Ada 2022 Glossary

A practical, developer-focused glossary of Ada terms that often mean something more precise (or different) than in other languages. This is intended as a printable supplement to the Ada Reference Manual glossary.

Updated: February 01, 2026

Key lifecycle terms (quick mapping)

- **Instantiation**: specialize a generic (compile-time concept).
- **Elaboration**: runtime startup for library units; runs initialization code before first use.
- **Allocation**: create a heap object via an allocator (new) from a storage pool.
- **Initialization**: give an object its initial state (default or explicit); occurs during elaboration, on block/subprogram entry, and during allocation.

Glossary

Term	Meaning
A	
Abort (task abort)	A request to prematurely stop a task. Semantics interact with rendezvous, protected operations, and abort-deferred regions.
Abstract type / subprogram	A tagged type that cannot be directly created (abstract), or a subprogram that must be overridden by descendants.
Access type	Ada's pointer type. Values designate objects (access-to-object) or subprograms (access-to-subprogram). Heap allocation uses an allocator (new) for access-to-object types.
Activation (task activation)	The runtime event where a task begins executing after being created and activated (often as a consequence of elaboration or scope entry).
Actual parameter (actual)	The argument supplied at a call site to match a formal parameter. Actuals may be passed positionally or by named association.

Term	Meaning
Aggregate	A value constructor for composite types (records/arrays). Ada 2022 adds and refines aggregate forms (notably delta aggregates).
Allocation	Creating an object in a storage pool (often the heap) via an allocator (new); allocation also initializes the new object.
Allocator	The syntactic form new T or new T'(...) that allocates storage (from an access type's pool) and initializes the object.
Aspect	A standardized way to attach properties, constraints, or contracts to entities using "with Aspect => ...".
Assertion	A condition checked at runtime (or proven), e.g., pragma Assert or contract aspects like Pre/Post/Invariant.
Attribute	An apostrophe-prefixed query or operation, e.g., T'First, X'Length, S'Image.
B	
Body	The implementation part of a package, subprogram, task, or protected unit.
Bounded error	A defined category of incorrect execution for which the implementation has bounded freedom (unlike erroneous execution).
C	
Call (dispatching call)	A call whose target is selected at runtime based on a tag (dynamic dispatch) for class-wide or tagged types.
Child unit	A library unit nested under another, e.g., package Parent.Child is ..., used for modularity and controlled visibility.
Class-wide type (T'Class)	A type that represents any type in the derivation class of T, enabling polymorphism and dispatching.
Clause (with/use)	with declares a dependency on a unit; use makes names (and possibly operators) directly visible.
Completion	The point where a declaration is completed by its full definition (e.g., incomplete or private types completed later).
Constant	An object that cannot be assigned after initialization (though it may designate mutable data via access types).
Constraint	A restriction on a subtype (range, discriminant, index, etc.).

Term	Meaning
Controlled type	A type with lifecycle hooks (Initialize/Adjust/Finalize) via Ada.Finalization; key to deterministic resource management.
D	
Declarative region	A scope where declarations live (package spec/body, subprogram, block, etc.), governing visibility and lifetime.
Declare block	A block statement with its own declarations and statements: declare ... begin ... end;
Deferred constant	A constant declared without its full value in one place and completed later (often in a package body).
Delta aggregate	Ada 2022 feature: build a new composite value by copying a base value and changing selected components.
Discriminant	A record parameter that can affect a type's constraints or shape (e.g., variant records).
Dispatching	Dynamic selection of a primitive operation for a tagged object based on its tag.
E	
Elaboration	Runtime startup for library units; executes elaboration code and performs library-level initialization before first use.
Elaboration order	The required order in which units are elaborated so dependencies are ready before use.
Entry	A synchronization point for tasks/protected objects; supports rendezvous/queued calls.
Exception	Ada's exception mechanism; exceptions can propagate, be handled, or terminate tasks/program.
F	
Finalization	Lifecycle cleanup for controlled objects; happens on scope exit and at program termination (and typically when heap objects are reclaimed).
Formal parameter	A parameter declared in a subprogram specification/profile (the signature).
Formal parameter identifier	The identifier used in the signature for a formal parameter (often casually called the parameter name).

Term	Meaning
Full view / partial view	Private types expose a partial view publicly and a full definition in the private part/body for encapsulation.
G	
Generic	A template (package/subprogram) parameterized by types, values, or subprograms.
Generic formal	A parameter of a generic (formal type, formal subprogram, formal object, etc.).
I	
Incomplete type	A type declared without its full definition yet; completed later (useful for mutual recursion and access types).
Initialization	Giving an object its initial state (default or explicit). Occurs during elaboration, on block/subprogram entry, and during allocation.
Instantiation	Supplying actuals to a generic to create a specialized unit (e.g., package P is new G (Integer);). This is not heap-object creation.
Invariant	A property that must hold for a type/object at defined points, often via aspects like Type_Invariant.
L	
Library unit	A separately compilable unit (package, subprogram, generic, etc.); central to Ada's elaboration model.
N	
Name resolution	Compile-time rules that determine which identifier/operator is meant, based on context, types, and visibility.
Named association	Mapping actuals to formals by name at a call site, e.g., Foo (X => 1, Y => 2).
Null	A no-op statement (null;) and a possible value for access types (null access).
O	
Overloading	Multiple entities share a name; Ada resolves by context, types, and visibility.
Override	Replacing an inherited primitive operation for a tagged type; Ada supports explicit overriding/not overriding indicators.

Term	Meaning
P	
Package	Ada's primary modular unit; has a spec (interface) and optionally a body (implementation).
Parallel loop	Ada 2022 supports parallel iteration constructs; exact execution strategy is implementation-defined within language rules.
Parameter mode	Directionality/permissions for a formal parameter: in, out, or in out.
Parameter profile (parameter specification)	The formal-parameter list of a subprogram declaration (its signature), including names, types, and modes.
Positional association	Mapping actuals to formals by position at a call site, e.g., Foo (1, 2).
Primitive operation	An operation associated with a type (especially tagged types) that participates in dispatching and inheritance.
Protected object	A concurrency construct that synchronizes access to shared data using protected procedures/functions/entries (monitor-like).
R	
Rendezvous	Task synchronization where an entry call is accepted by another task via an accept statement.
S	
Separate compilation	Ada's compilation model: units are compiled independently, then bound/linked with elaboration ordering.
Specification (spec)	The interface part of a package or subprogram; contrasted with the body.
Storage pool	The allocator backing for access types; new allocates from a pool (heap by default, but customizable).
Subtype	A type view plus constraints (e.g., Integer range 1 .. 10); central to Ada's safety model.
T	
Tagged type	An OOP type supporting inheritance and dispatching (has a runtime tag).
Task	Ada's built-in concurrency abstraction; tasks may have entries for rendezvous.

Term	Meaning
Type extension	Creating a new tagged type derived from another and adding components.
U	
Unchecked_Deallocation	A standard generic for explicitly freeing heap storage; dangerous if misused and interacts with finalization.
Use clause	Brings names (including operators) from a package into direct visibility; convenient but can obscure origins.
V	
Variant record	A record whose components depend on discriminants (variant parts).
Visibility	What names are directly usable in a scope; controlled by regions, with/use, private parts, and child units.
W	
With clause	Declares a dependency on another unit (imports it).